

Otoczka wypukła dla zbioru punktów w przestrzeni dwuwymiarowej

Konrad Idzik

Kacper Wąchała

1. Cel projektu

Celem projektu jest implementacja algorytmów służących do wyznaczania otoczki wypukłej. W ramach projektu zostaną zaimplementowane następujące algorytmy:

- Algorytm Grahama
- Algorytm Jarvisa
- Algorytm górnej i dolnej otoczki
- Algorytm przyrostowy
- Algorytm Quickhull
- Algorytm dziel i rządź
- Algorytm Chan'a

Projekt uwzględnia również:

- **Wizualizację etapów działania algorytmów**, umożliwiając graficzne przedstawienie procesu konstrukcji otoczki wypukłej.
- **Analizę i porównanie efektywności algorytmów na różnych zbiorach testowych**, uwzględniając różne przypadki rozmieszczenia punktów.

2. Specyfikacja

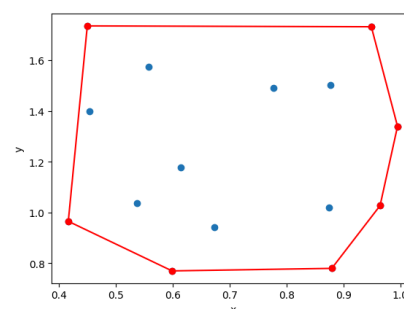
Program został uruchomiony na systemie Linux (dystrybucja Pop! OS) z procesorem 12th Gen Intel i7-12700H. Całość została napisana w Pythonie przy użyciu środowiska Jupyter Notebook. Do realizacji projektu wykorzystano popularne biblioteki zewnętrzne, takie jak Matplotlib, Pandas i NumPy. Wizualizacja wyników została zrealizowana za pomocą narzędzia opracowanego przez studentów z koła naukowego BIT.

3. Definicja otoczki

Wielokąt wypukły jest **otoczką wypukłą** zbioru punktów $S \Leftrightarrow$ gdy jest najmniejszym wielokątem wypukłym zawierającym zbiór S .

Przykładowa wizualizacja otoczki wypukłej jest przedstawiona na Rysunku 1

Obliczenie otoczki polega na wyznaczeniu listy punktów należących do otoczki w kierunku przeciwnym do wskazówek zegara.



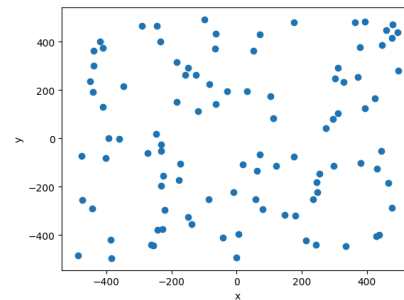
Rysunek 1: Przykładowa otoczką wypukłą

4. Analizowane zbiory punktów

W dalszej analizie wykorzystane zostaną zbiory n -punktowe, zależne od parametru n , aby zbadać wpływ jego zmiany na wydajność algorytmów. Każdy zbiór będzie reprezentował określoną konfigurację n punktów oraz spełniał konkretne założenia. W przeprowadzonym projekcie dokonano generacji zbiorów punktów przy użyciu funkcji `random.uniform()` oraz `random.random()` z biblioteki Numpy.

4.1. Zbiory A n-punktowe

Zbiory zawierające n losowo wygenerowanych punktów takich, takich że $(x, y) \in [-500, 500]^2$. Rysunek 2 przedstawia zbiór A, gdy $n = 100$.



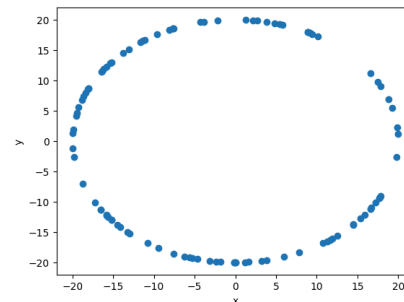
Rysunek 2: Zbiór A 100-punktowy

4.2. Zbiory B n-punktowe

Zbiory składające się z n losowo wygenerowanych punktów, które znajdują się na okręgu o środku $(0, 0)$ i promieniu 20. Punkty są wygenerowane za pomocą równania parametrycznego okręgu:

$$\begin{cases} x(t) = R \cos(t) \\ y(t) = R \sin(t) \end{cases} \quad (1)$$

Jest to przypadek, w którym wszystkie punkty należą do otoczki wypukłej, dlatego obliczona otoczka powinna zawierać wszystkie n punktów. Rysunek 3 przedstawia wizualizację przykładowego zbioru B 100-punktowego.



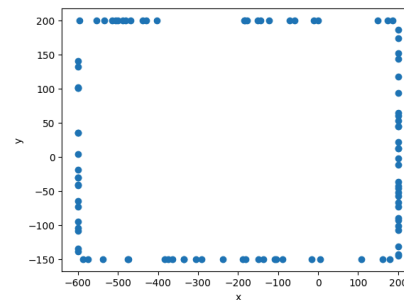
Rysunek 3: Zbiór B 100-punktowy

4.3. Zbiory C n-punktowe

Zbiory składające się z n punktów leżących na bokach prostokąta o wierzchołkach $(-600, -150)$, $(200, -150)$, $(200, 200)$, $(-600, 200)$. Przypadek ten służy do przetestowania działania algorytmów w sytuacji, gdy istnieją punkty współliniowe, sprawdzając, czy algorytmy poprawnie eliminują je z otoczki wypukłej.

Dodatkowo wiadomo, że liczba punktów należących do otoczki wypukłej, oznaczona jako m , spełnia warunek $4 \leq m \leq 8$. Wynika to z faktu, że jeżeli żaden punkt nie leży w wierzchołku prostokąta, to na każdym boku muszą znajdować się przynajmniej dwa punkty, aby poprawnie wyznaczyć otoczkę.

Rysunek 4 przedstawia przykładowy Zbiór C 100-punktowy



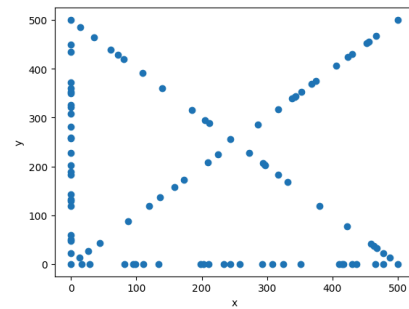
Rysunek 4: Zbiór C 100-punktowy

4.4. Zbiory D n-punktowe

Zbiór składa się z n punktów rozmieszczonych na kwadracie o wierzchołkach $(0,0)$, $(500,0)$, $(500,500)$, $(0,500)$ oraz na jego przekątnych. Punkty są generowane zgodnie z następującymi zasadami:

- Wierzchołki kwadratu zawsze należą do zbioru (4 punkty).
- Punkty na dwóch bokach kwadratu są generowane losowo w liczbie $\lfloor \frac{n-4}{2} \rfloor$, co odpowiada połowie punktów innych niż wierzchołki.
- Punkty na dwóch przekątnych kwadratu są również generowane losowo w liczbie $\lfloor \frac{n-4}{2} \rfloor$, co odpowiada drugiej połowie punktów innych niż wierzchołki.

W ten sposób stosunek liczby punktów na przekątnych do liczby punktów na bokach wynosi 1:1. Rysunek 4 przedstawia przykładowy Zbiór D 100-punktowy



Rysunek 5: Zbiór D 100-punktowy

5. Narzędzie do wprowadzania punktów

Program oferuje również możliwość wprowadzania punktów przez użytkownika za pomocą interfejsu graficznego. Dzięki temu użytkownik może przetestować działanie poniższych algorytmów na własnym zbiorze punktów oraz zobaczyć ich wizualizacje. Takie rozwiązanie umożliwia lepsze zrozumienie działania algorytmów.

6. Opisy algorytmów oraz metodologia

6.1. Funkcja det

W poniższych algorytmach do określenia, po której stronie danej prostej znajduje się punkt, wykorzystano funkcję zdefiniowaną wzorem:

$$\det(a, b, c) = \det \begin{pmatrix} a_x - c_x & a_y - c_y \\ b_x - c_x & b_y - c_y \end{pmatrix} \quad (2)$$

Zastosowano wyznacznik macierzy 2×2 , ponieważ w praktyce okazał się bardziej efektywny niż wyznacznik macierzy 3×3 w testowanych przypadkach. Wykorzystanie prostszego wyznacznika zmniejszyło liczbę operacji oraz poprawiło wydajność obliczeń. Przyjęto tolerancję dla wartości zerowej na poziomie $\varepsilon = 10^{-24}$.

6.2. Algorytm Grahama

6.2.1. Opis

Algorytm Grahama opiera się na sortowaniu punktów względem kąta jaki tworzą z punktem p_0 o najmniejszej wartości współrzędnej y i dodatnim kierunkiem osi x , a następnie liniowym sprawdzeniu, czy każde trzy kolejne punkty tworzą skręt lewostronny.

1. **Wybór punktu p_0 :** Wybieramy punkt p_0 , który ma najmniejszą współrzędną y (jeśli jest ich więcej, wybieramy ten o najmniejszej współrzędnej x).

2. **Sortowanie punktów:** Wszystkie pozostałe punkty sortujemy względem kąta jaki tworzą z punktem p_0 i dodatnim kierunkiem osi x . Jeśli więcej niż jeden punkt tworzy ten sam kąt, usuwane są wszystkie z nich z wyjątkiem najbardziej oddalonego od p_0 .
3. **Inicjalizacja stosu:** Do stosu (np. listy) dodajemy pierwsze trzy punkty z posortowanego zbioru.
4. **Iteracja po punktach:** Przechodzimy przez punkty w posortowanej kolejności od ostatniego do pierwszego:
 - Dla każdego punktu sprawdzamy, czy dwa ostatnie punkty ze stosu tworzą z nim skręt lewostronny:
 - Dopóki **nie tworzą** skrętu lewostronnego, usuwamy kolejno punkty ze stosu.
 - W innym przypadku, do stosu dodawany jest rozpatrywany punkt.

Algorytm zwraca punkty otoczki zadane w kierunku przeciwnym do ruchu wskazówek zegara.

6.2.2. Złożoność czasowa

Sortowanie punktów względem kąta przebiega w czasie $O(n \log n)$. Złożoność operacji przejścia po kolejnych punktach i modyfikacji stosu jest rzędu $O(n)$. Zatem całkowitą złożoność algorytmu szacuje się na $O(n \log n)$.

6.2.3. Metodologia

6.2.3.1. Sortowanie względem kąta

Procedura sortowania wykorzystuje wbudowaną metodę `sort()` z niestandardowym komparatorem, który porównuje punkty względem kąta jaki tworzą z p_0 i dodatnim kierunkiem osi x . Komparator wykorzystuje znak wyznacznika w celu określenia relacji między punktami.

6.3. Algorytm Jarvisa

6.3.1. Opis

Algorytm Jarvisa działa na zasadzie dodawania do otoczki kolejnych punktów, które tworzą z ostatnią krawędzią otoczki najmniejszy kąt.

1. **Inicjalizacja:** Tworzona jest struktura danych (np. lista), która przechowuje punkty należące do dolnej otoczki w kolejności przeciwnym do ruchu wskazówek zegara.
2. **Wybór punktu p_0 :** Wybieramy punkt p_0 , który ma najmniejszą współrzędną y (jeśli jest ich więcej, wybieramy ten o najmniejszej współrzędnej x).
3. **Budowanie otoczki:** Dla każdego kolejnego punktu p_i należącego do otoczki szukamy punktu p_{i+1} , dla którego kąt jaki tworzy z ostatnią krawędzią otoczki liczony w kierunku przeciwnym do ruchu wskazówek zegara jest najmniejszy. Znaleziony punkt p_{i+1} dodajemy do otoczki.

6.3.2. Złożoność czasowa

Złożoność czasowa algorytmu Jarvisa wynosi $O(nh)$, gdzie h oznacza liczbę punktów w otoczce, ponieważ operacja wyszukiwania punktu tworzącego najmniejszy kąt z ostatnią krawędzią otoczki ma złożoność $O(n)$ i jest wykonywana dla każdego punktu należącego do otoczki.

6.4. Algorytm górnej i dolnej otoczki

6.4.1. Opis

Algorytm górnej i dolnej otoczki opiera się na obserwacji, że podczas przechodzenia wzdłuż punktów otoczki wypukłej, w kolejności rosnących współrzędnych x , trzy kolejne wierzchołki dolnego łańcucha muszą tworzyć skręt lewostronny. Analogicznie, dla górnego łańcucha, skręt lewostronny musi być spełniony przy przechodzeniu po punktach w kolejności malejących współrzędnych x .

1. **Sortowanie punktów:** Wszystkie punkty w zbiorze S są sortowane według współrzędnej x (jeśli x są równe, według y).

2. Budowanie dolnego łańcucha:

1. **Inicjalizacja:** Tworzona jest struktura danych (np. lista), która przechowuje punkty należące do dolnej otoczki w kolejności przeciwnym do ruchu wskazówek zegara.
2. **Iteracja po punktach:** Przechodzimy przez punkty w posortowanej kolejności od pierwszego do ostatniego:
 - Dla każdego punktu sprawdzamy, czy nowy punkt wraz z dwoma ostatnimi punktami w łańcuchu nie tworzy skrętu lewostronnego.
 - Jeśli **nie tworzy skrętu lewostronnego**, usuwamy ostatni punkt z dolnego łańcucha, ponieważ nie może należeć do otoczki.
 - W innym przypadku, dodawany jest nowy punkt do dolnego łańcucha

3. Budowanie górnego łańcucha:

1. **Inicjalizacja:** Tworzona jest struktura danych (np. lista), która przechowuje punkty należące do górnej otoczki w kolejności przeciwnym do ruchu wskazówek zegara.
2. **Iteracja po punktach:** Przechodzimy przez punkty w posortowanej kolejności od ostatniego do pierwszego:
 - Dla każdego punktu sprawdzamy, czy nowy punkt wraz z dwoma ostatnimi punktami w łańcuchu nie tworzy skrętu lewostronnego.
 - Jeśli **nie tworzy skrętu lewostronnego**, usuwany jest ostatni punkt z górnego łańcucha, ponieważ nie może należeć do otoczki.
 - W innym przypadku, dodawany jest nowy punkt do górnego łańcucha
4. **Łączenie górnego i dolnego łańcucha:** Punkty z dolnej otoczki i górnej otoczki są łączone, tworząc pełną otoczkę wypukłą.

Można konstruować łańcuchy, korzystając z faktu, że trzy kolejne wierzchołki górnego łańcucha muszą tworzyć skręt prawostronny. Analogicznie, dla dolnego łańcucha, skręt prawostronny musi być spełniony przy przechodzeniu po punktach w kolejności malejących współrzędnych x . Jednak przy łączeniu górnego i dolnego łańcucha konieczne jest odwrócenie kolejności punktów w górnym i dolnym łańcuchu, tak aby zachować kolejność punktów należących do otoczki wypukłej w kierunku przeciwnym do ruchu wskazówek zegara.

6.4.2. Złożoność czasowa

Sortowanie punktów odbywa się w czasie $O(n \log n)$. Następnie budowa dolnego i górnego łańcucha wymaga jednokrotnego przejścia przez wszystkie punkty, co ma złożoność $O(n)$. W konsekwencji całkowita złożoność algorytmu wynosi $O(n \log n)$, gdzie etap sortowania dominuje nad etapem budowania łańcuchów.

6.5. Algorytm przyrostowy

6.5.1. Opis

Algorytm przyrostowy działa na zasadzie stopniowego dodawania kolejnych nieleżących w otoczce punktów poprzez znajdowanie stycznych do otoczki. W celu wyeliminowania konieczności sprawdzania położenia punktu względem otoczki, dane są sortowane. Aby sprawnie modyfikować otoczkę dokonano podziału na dwa łańcuchy — górny i dolny. Dla każdego punktu algorytm sprawdza, z którymi punktami z górnego łańcucha tworzy skręt lewostronny, a z którymi z dolnego łańcucha tworzy skręt prawostronny.

1. **Sortowanie punktów:** Wszystkie punkty w zbiorze S są sortowane według współrzędnej x (jeśli x są równe, według y).
2. **Inicjalizacja łańcuchów:** Tworzone są dwie struktury danych (np. listy), które przechowują punkty należące do łańcuchów w kolejności rosnących współrzędnych x .

3. **Iteracja po punktach:** Przechodzimy przez punkty w posortowanej kolejności od drugiego do ostatniego:

- Dla każdego punktu sprawdzamy czy:
 - Tworzy on wraz z dwoma ostatnimi punktami górnego łańcucha skręt lewostronny:
 - Jeśli **nie tworzy** skrętu lewostronnego, usuwany jest ostatni punkt z górnego łańcucha.
 - W innym przypadku, dodajemy dany punkt do górnego łańcucha
 - Tworzy on wraz z dwoma ostatnimi punktami dolnego łańcucha skręt prawostronny:
 - Jeśli **nie tworzy** skrętu prawostronnego, usuwany jest ostatni punkt z dolnego łańcucha.
 - W innym przypadku, dodajemy dany punkt do dolnego łańcucha.

4. **Łączenie górnego i dolnego łańcucha:** Punkty z obu łańcuchów są łączone ze sobą liniowo w otoczkę.

6.5.2. Złożoność

Sortowanie punktów przebiega w czasie $O(n \log n)$. Operacja przejścia przez wszystkie punkty zbioru i aktualizacji obu łańcuchów ma złożoność rzędu $O(n)$. Łączenie łańcuchów odbywa się w czasie $O(n)$. Zatem wynikowa złożoność algorytmu wynosi $O(n \log n)$.

6.6. Algorytm Quickhull

6.6.1. Opis

Algorytm Quickhull wykorzystuje podejście dziel i rządź, podobnie jak algorytm Quicksort, choć jego celem jest wyznaczenie otoczki wypukłej, a nie sortowanie. W obu algorytmach kluczowym elementem jest podział problemu na mniejsze pod problemy, co pozwala na efektywne rozwiązanie całego problemu. Algorytm przyjmuje zbiór punktów S oraz punkty skrajne A i B . Kroki algorytmu:

1. **Wyszukiwanie kolejnego punktu należącego do otoczki:** Dla danej pary punktów A i B , algorytm szuka punktu C , który jest najbardziej oddalony od prostej łączącej te punkty. Punkt C jest dodawany do otoczki pomiędzy punktami A i B .
2. **Podział zbioru S na dwa podzbiory:**
 - Punkty znajdujące się po lewej stronie prostej AC
 - Punkty znajdujące się po lewej stronie prostej CB
3. **Rekurencyjne wywołanie:** Dla każdego z dwóch nowo utworzonych podzbiorów algorytm powtarza kroki 1 i 2: Dla każdego podzbioru wyszukiwany jest punkt najbardziej oddalony od odpowiedniej prostej, a następnie zbiór jest ponownie dzielony na mniejsze podzbiory.

Aby wyznaczyć otoczkę wypukłą powyższym algorytmem. Należy wykonać następujące kroki:

1. **Wyszukiwanie punktów ekstremalnych:** Znajdź punkty A i B , które mają odpowiednio minimalną i maksymalną wartość współrzędnej x . Punkty te zawsze należą do otoczki wypukłej. W przypadku, gdy istnieje więcej punktów o tej samej minimalnej lub maksymalnej współrzędnej x , wybierz punkt o odpowiednio minimalnej lub maksymalnej wartości współrzędnej y .
2. **Podział na dwie części:** Podziel zbiór punktów S na dwie części:
 - Punkty znajdujące się po lewej stronie prostej AB ,
 - Punkty znajdujące się po prawej stronie prostej AB .
3. **Tworzenie otoczki:** Uruchom powyższy algorytm (kroki 1–3) rekurencyjnie dla obu części zbioru punktów:
 - Najpierw znajdź punkty należące do otoczki dla punktów znajdujących się po lewej stronie prostej AB ,
 - Następnie znajdź punkty należące do otoczki dla punktów znajdujących się po prawej stronie prostej AB .

Wynikiem algorytmu jest lista punktów tworzących otoczkę wypukłą, uporządkowanych zgodnie z ruchem przeciwnym do wskazówek zegara.

6.6.2. Złożoność czasowa

Złożoność algorytmu Quickhull w najgorszym przypadku wynosi $O(n^2)$, gdy punkty są niekorzystnie rozmieszczone, np. wszystkie leżą na otoczce. W optymistycznym przypadku, przy równomiernym podziale punktów, złożoność wynosi $O(n \log n)$.

6.6.3. Metodologia

6.6.3.1. Funkcja dzieląca zbiór względem prostej

Funkcja przyjmuje zbiór S oraz punkty a i b , dla każdego punktu $c \in S$ oblicza wartość wyznacznika $\det(a, b, c)$, określając położenie c względem prostej wyznaczonej przez punkty a i b . Punkty o dodatnim wyniku wyznacznika są dodawane do zbioru po lewej stronie prostej, natomiast punkty o wyniku ujemnym trafiają do zbioru po prawej stronie.

6.6.3.2. Funkcja szukająca najbardziej oddalonego punktu

Funkcja przyjmuje zbiór S oraz punkty a i b , wyznaczające prostą, względem której obliczana jest odległość punktów ze zbioru S . Dla każdego punktu $c \in S$, funkcja oblicza wartość wyznacznika $\det(a, b, c)$ i zwraca punkt c , dla którego wartość wyznacznika jest największa. Algorytm korzysta z własności, że $\det(a, b, c)$ jest równy polu równoległoboku rozpiętego na wektorach $\overrightarrow{AB}$ i $\overrightarrow{AC}$. Pole trójkąta ABC można zatem wyrazić jako:

$$P = \frac{\det(a, b, c)}{2} \quad (3)$$

Wykorzystując wzór na pole trójkąta:

$$\begin{aligned} \frac{1}{2} |AB| h &= \frac{\det(a, b, c)}{2} \\ h &= \frac{\det(a, b, c)}{|AB|} \end{aligned} \quad (4)$$

Ponieważ $|AB|$ jest stałe dla każdego $c \in S$

$$h \sim \det(a, b, c) \quad (5)$$

Wynika z tego, że odległość punktu c od prostej AB jest proporcjonalna do wartości wyznacznika $\det(a, b, c)$. W związku z tym, punkt, dla którego wyznacznik osiąga największą wartość, jest najbardziej oddalony od prostej AB .

6.6.3.3. Quickhull

Algorytm jest zgodny z opisem w Sekcji 6.6.1, jednak różni się pod względem dodawania punktów do otoczki wypukłej. W przypadku implementacji w Pythonie optymalne dodawanie punktów pomiędzy dwoma elementami listy może być trudne do osiągnięcia. Dla standardowej listy w Pythonie, operacja wstawiania elementu w konkretnym miejscu ma złożoność $O(n)$, ponieważ w najgorszym przypadku konieczne jest przesunięcie wszystkich elementów znajdujących się po wstawianym punkcie.

W związku z tym, punkty należące do otoczki wypukłej w algorytmie Quickhull są dodawane na koniec listy, co zapewnia złożoność $O(1)$ dla każdej operacji dodawania punktu. Niemniej jednak, punkty te nie są ułożone w kierunku przeciwnym do wskazówek zegara.

Aby naprawić kolejność punktów otoczki wypukłej w algorytmie Quickhull, po zakończeniu procesu jej wyznaczania wykonujemy następujące kroki, przypominające algorytm Grahama:

- **Wybór punktu p_0 :** Wybieramy punkt p_0 , który ma najmniejszą współrzędną y (jeśli jest ich więcej, wybieramy ten o najmniejszej współrzędnej x).
- **Usunięcie punktu p_0 :** Usuujemy punkt p_0 z listy punktów otoczki, aby uniknąć jego uwzględniania w dalszym etapie sortowania.
- **Sortowanie punktów:** Sortujemy pozostałe punkty otoczki względem kąta, jaki tworzą z punktem p_0 wykorzystując wyznacznik
- **Dodanie p_0 na koniec otoczki:** Po posortowaniu punktów w odpowiedniej kolejności, dodajemy punkt p_0 z na koniec listy punktów otoczki, przywracając pełny cykl w kierunku przeciwnym do wskazówek zegara.

W wyniku tych operacji punkty otoczki będą uporządkowane w kolejności przeciwnej do wskazówek zegara.

6.7. Algorytm dziel i rządź

6.7.1. Opis

Algorytm, jak nazwa sugeruje, wykorzystuje rekurencyjne podejście do problemu, analogicznie do np. algorytmu Mergesort. W celu sprawnej modyfikacji otoczki podzielono ją na dwa łańcuchy — górny oraz dolny. Algorytm składa się zatem z wywołań dwóch funkcji rekurencyjnych tworzących łańcuchy: `upper_hull` i `lower_hull`, które za argument przyjmują zbiór S .

1. **Sortowanie punktów:** Wszystkie punkty w zbiorze S są sortowane według współrzędnej x (jeśli x są równe, według y).
2. **Inicjalizacja łańcuchów:** Tworzone są dwie struktury danych (np. listy), które przechowują punkty należące do łańcuchów w kolejności rosnących współrzędnych x .
3. **Wywołanie funkcji:**
 - Funkcja `upper_hull`:
 1. **Podział zbioru S na dwa podzbiory:** Punkty ze zbioru S są dzielone względem mediany na dwa podzbiory A i B o podobnej liczności.
 2. **Wyznaczenie otoczek górnych w podzbiorach:** Algorytm rekurencyjnie wyznacza otoczki dla każdego z podzbiorów A oraz B .
 3. **Scalanie funkcją `merge_upper`:** Dopóki nie jest wyznaczony cały górny łańcuch, algorytm scala otoczki w podzbiorach w kolejności odwrotnej do podziału zbioru S .
 - Funkcja `lower_hull`:
 1. **Podział zbioru S na dwa podzbiory:** Punkty ze zbioru S są dzielone względem mediany na dwa podzbiory A i B o podobnej liczności.
 2. **Wyznaczenie otoczek dolnych w podzbiorach:** Algorytm rekurencyjnie wyznacza otoczki dla każdego z podzbiorów A oraz B .
 3. **Scalanie funkcją `merge_lower`:** Dopóki nie jest wyznaczony cały dolny łańcuch, algorytm scala sąsiednie otoczki w podzbiorach w kolejności odwrotnej do podziału zbioru S .
4. **Łączenie górnego i dolnego łańcucha:** Punkty z obu łańcuchów są ze sobą łączone liniowo w otoczkę

6.7.2. Złożoność czasowa

Sortowanie punktów przebiega w czasie $O(n \log n)$. Operacja podziału i scalania otoczek ma złożoność $O(n \log n)$. Zatem wynikowa złożoność algorytmu dziel i rządź wynosi $O(n \log n)$

6.7.3. Metodologia

6.7.3.1. Algorytm scalania sąsiednich otoczek

Jest to algorytm realizujący operację scalania w czasie $O(n)$. W głównym algorytmie jest podzielony na dwie osobne funkcje operujące na różnych łańcuchach. Zakładamy że podzbiór A zawiera punkty o mniejszej wartości współrzędnej x , a podzbiór B — o większej. Dodatkowo przyjmijmy, że:

- p_i — punkt o indeksie i z podzbioru A o maksymalnej współrzędnej x .
- q_j — punkt o indeksie j z podzbioru B o minimalnej współrzędnej x .

Funkcje działają następująco:

- Funkcja `merge_upper`:
 1. Dopóki $(p_{i-1}$ istnieje i (p_{i-1}, p_i, q_j) nie tworzą skrętu prawostronnego) lub $(q_{j+1}$ istnieje i (p_i, q_j, q_{j+1}) nie tworzą skrętu prawostronnego):
 - Jeśli p_{i-1} istnieje i (p_{i-1}, p_i, q_j) nie tworzą skrętu prawostronnego:
 - $i \leftarrow i - 1$
 - Jeśli q_{j+1} istnieje i (p_i, q_j, q_{j+1}) nie tworzą skrętu prawostronnego:
 - $j \leftarrow j + 1$
 2. Zwróć jeden łańcuch połączony odcinkiem (p_i, q_j) .
- Funkcja `merge_lower`:
 1. Dopóki $(p_{i-1}$ istnieje i (p_{i-1}, p_i, q_j) nie tworzą skrętu lewostronnego) lub $(q_{j+1}$ istnieje i (p_i, q_j, q_{j+1}) nie tworzą skrętu lewostronnego):
 - Jeśli p_{i-1} istnieje i (p_{i-1}, p_i, q_j) nie tworzą skrętu lewostronnego:
 - $i \leftarrow i - 1$
 - Jeśli q_{j+1} istnieje i (p_i, q_j, q_{j+1}) nie tworzą skrętu lewostronnego:
 - $j \leftarrow j + 1$
 2. Zwróć jeden łańcuch połączony odcinkiem (p_i, q_j) .

6.8. Algorytm Chan'a

6.8.1. Opis

Algorytm łączy idee algorytmu Grahama i Jarvisa, aby osiągnąć złożoność $O(n \log h)$, gdzie h to liczba punktów na otoczce wypukłej. Najpierw zbiór S o rozmiarze n dzielony jest na $r = \lceil \frac{n}{m} \rceil$ grup, każda z grup zawiera m punktów (ostatnia grupa może zawierać mniej punktów). Następnie dla każdej grupy obliczana jest otoczka wypukła przy użyciu algorytmu Grahama, co prowadzi do powstania m lokalnych otoczek. Kolejnym krokiem jest zastosowanie algorytmu Jarvisa do połączenia tych lokalnych otoczek w jedną globalną otoczkę wypukłą. Wykorzystywana jest optymalna metoda wyznaczania stycznej z punktu do otoczki wypukłej. Algorytm `PartialHull` przy danym m :

1. Oblicz $r = \lceil n/m \rceil$. Podziel zbiór P na rozłączne podzbiory $P_1, P_2, \dots, P_k$, takie że $\bigcup_{i=1}^k P_i = P$, a każdy podzbiór ma co najwyżej m punktów.
2. Dla $i = 1$ do r : Oblicz otoczkę wypukłą $CH(P_i)$ za pomocą algorytmu Grahama i przechowaj wierzchołki w uporządkowanej tablicy.
3. Ustaw $p_0 = (-\infty, 0)$, a p_1 jako punkt o najmniejszej współrzędnej y w P .
4. Dla $j = 1$ do m :
 1. Dla $i = 1$ do r : Oblicz punkt $q_i \in P_i$, który maksymalizuje kąt $\angle p_{j-1} p_j q_i$
 2. Ustaw p_{j+1} punkt $q \in \{q_1, q_2, \dots, q_r\}$, który maksymalizuje kąt $\angle p_{j-1} p_j q$
 3. Jeśli $p_{j+1} = p_1$, to zwróć $\langle p_1, \dots, p_j \rangle$
5. Zwróć komunikat „ m za małe”, jeśli otoczka nie została wyznaczona.

Kolejnym pytaniem jest, jak efektywnie dobrać odpowiednią wartość m . Sprawdzanie kolejnych wartości m w sposób liniowy zaburzyłoby oczekiwaną złożoność algorytmu. Algorytm Chana stosuje sprytniejsze podejście, w którym wartości m są zwiększane w szybkim tempie. Wartości te są wyznaczone według wzoru:

$$m = \min(n, 2^{2^t}) \quad (6)$$

gdzie t to kolejne liczby naturalne. Sprawdzamy kolejne m dopóki, dopóty uda się zbudować otoczkę wypukłą. Kroki algorytmu Hull:

1. Dla $t = 1, 2, 3, \dots$
 1. Oblicz $m = \min(n, 2^{2^t})$
 2. Wywołaj funkcję PartialHull(P, m) i zapisz wynik w L .
 3. Jeśli $L \neq m$ za małe, zwróć L .

6.8.2. Złożoność czasowa

6.8.2.1. PartialHull

Dla każdego rozłącznego zbioru P_i uruchamiany jest algorytm Grahama, którego złożoność wynosi $O(m \log m)$, gdzie m to maksymalna liczba punktów w każdej z grup. Ponieważ mamy r grup, łączna złożoność wynosi $O(rm \log m) = O(n \log m)$.

Następnie algorytm Jarvisa działa dla r grup. Przy wyznaczaniu kolejnego punktu w algorytmie Jarvisa musimy znaleźć styczną z danego punktu do każdej z r otoczek, z których każda zawiera m punktów. Wyznaczenie stycznej dla jednej otoczki ma złożoność $O(\log m)$, więc dla wszystkich r otoczek łączna złożoność wynosi $O(r \log m)$. Po wyznaczeniu stycznych, należy określić, który z punktów styczności tworzy najmniejszy kąt z ostatnią dodaną krawędzią otoczki, operacja ta ma złożoność $O(r)$. Łączna złożoność tej części algorytmu wynosi zatem $O(kr \log m) = O(\frac{hn}{m} \log m)$.

Podsumowując PartialHull ma złożoność $O((n + \frac{hn}{m}) \log m)$. Dla $m = h$ złożoność wynosi $O(n \log h)$.

6.8.2.2. Hull

Algorytm kończy swoje działanie, gdy uda się wyznaczyć otoczkę wypukłą. Ostateczna wartość t spełnia zależność $t = \lceil \log(\log h) \rceil$.

W najgorszym przypadku, gdy $h = n$, ostatnie t wynosi $t = \lceil \log(\log n) \rceil$. Złożoność wynosi:

$$\sum_{t=1}^{\log(\log(h))} O(n \log 2^{2^t}) = \sum_{t=1}^{\log(\log(h))} O(n 2^t) \leq O(n 2^{1+\log(\log h)}) = O(n \log h) \quad (7)$$

6.8.3. Metodologia

6.8.3.1. Algorytm znajdowania stycznej z punkty do otoczki

Algorytm ten musi posiadać złożoność $O(\log m)$, gdzie m to liczba wierzchołków w otoczce wypukłej. Problem można sprowadzić do wyszukiwania ekstremalnego punktu, wprowadzając odpowiednią relację porządku między punktami. Rozważmy punkt p , względem którego szukamy stycznej, oraz dwa punkty v_i i v_j , które należą do otoczki wypukłej.

Wprowadzamy następującą relację:

$$v_i < v_j \Leftrightarrow \det(p, v_i, v_j) < 0 \quad (8)$$

Relacja ta oznacza, że punkt v_j jest większy od v_i , jeśli znajduje się po prawej stronie prostej przechodzącej przez p oraz v_i . Dzięki takiej definicji, punkt na otoczce wypukłej, przez który przechodzi poszukiwana styczna, będzie punktem maksymalnym względem tej relacji. Taki punkt będzie miał dwóch sąsiadów, którzy są od niego mniejsi według tej relacji.

Krawędź e_i nazywamy **rosnącą** jeżeli $v_i < v_{i+1}$ lub gdy $\det(p, v_i, v_{i+1}) = 0$ co odpowiada przypadkowi, gdy styczna pokrywa się z krawędzią otoczki wypukłej.

Oznaczenie takiej krawędzi jako rosnącej pozwala nam wybrać ostatni punkt należący do tej krawędzi.

Krawędź e_i nazywamy **malejącą** jeżeli $v_i > v_{i+1}$

Podejście to umożliwia efektywne wykorzystanie wyszukiwania binarnego do znalezienia punktu stycznego w czasie $O(\log m)$

Aby znaleźć punkt ekstremalny, stosujemy algorytm wyszukiwania binarnego. Na początku ustawiamy $L = 0$ i $R = m$. Sprawdzamy, czy wierzchołek v_0 jest ekstremum, oceniając, czy jego sąsiednie wierzchołki są mniejsze względem określonej relacji porządku. Jeśli v_0 nie jest ekstremum, obliczamy $C = \frac{R-L}{2}$, a następnie sprawdzamy, czy wierzchołek v_C spełnia kryterium ekstremum. Jeśli v_C również nie jest ekstremum, zawężamy poszukiwania do jednego z dwóch przedziałów: $[L, C]$ lub $[C, R]$, w zależności od krawędzi e_C oraz e_L (Tabela 1).

Krawędź e_L	Krawędź e_C	Sytuacja	Wybór przedziału
Rosnąca	Malejąca	-	$[L, C]$
	Rosnąca	$v_C > v_L$	$[C, R]$
	Rosnąca	$v_C < v_L$	$[L, C]$
Malejąca	Rosnąca	-	$[C, R]$
	Malejąca	$v_C < v_L$	$[C, R]$
	Rosnąca	$v_C > v_L$	$[L, C]$

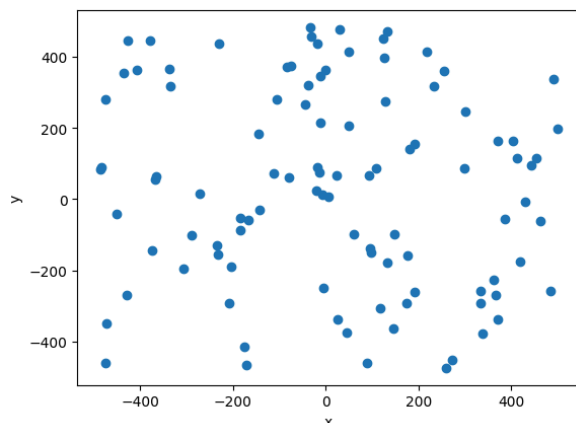
Tabela 1: Sposób wyboru przedziału

Proces powtarzamy, aż znajdziemy punkt ekstremalny.

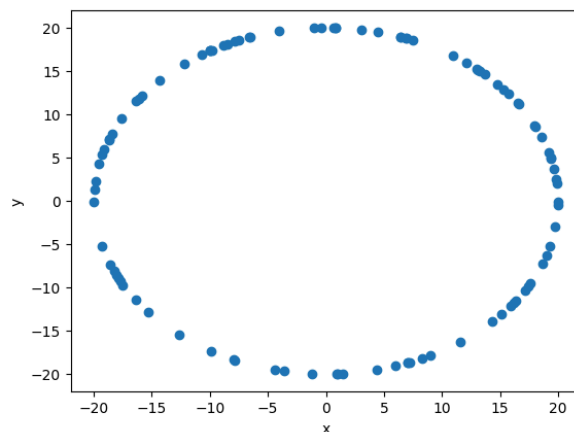
7. Test algorytmów

Powyższe algorytmy zostały przetestowane na zestawach danych przygotowanych przez Koło Naukowe BIT. Wszystkie zwróciły poprawne wyniki, co potwierdza ich zgodność z założeniami teoretycznymi oraz poprawność implementacji. Dodatkowo przeprowadzono testy sprawdzające, czy różne algorytmy generują identyczne wyniki na poniższych zestawach danych:

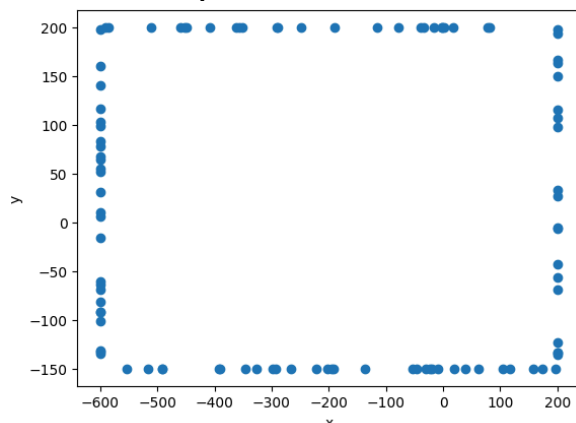
- Zbiór 1, który należy do zbiorów A 100-punktowych (Rysunek 6)
- Zbiór 2, który należy do zbiorów B 100-punktowych (Rysunek 7)
- Zbiór 3, który należy do zbiorów C 100-punktowych (Rysunek 8)
- Zbiór 4, który należy do zbiorów D 100-punktowych (Rysunek 9)



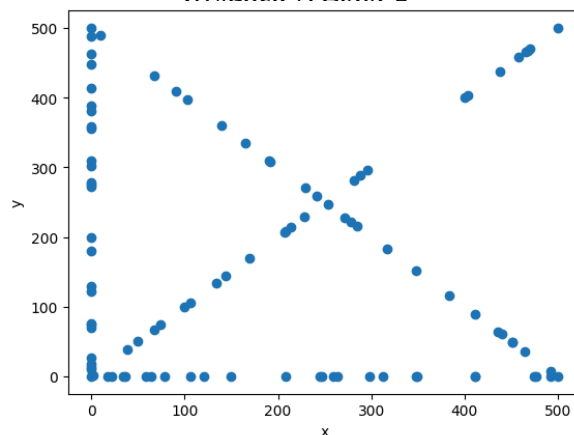
Rysunek 6: Zbiór 1



Rysunek 7: Zbiór 2



Rysunek 8: Zbiór 3



Rysunek 9: Zbiór 4

7.1. Uzyskane wyniki

Dla powyższych zbiorów danych przeprowadzono testy wszystkich analizowanych algorytmów. W poniższej tabeli przedstawiono liczbę punktów należących do otoczki wypukłej, wyznaczonej przez każdy z tych algorytmów.

Zbiór	Algorytm Grahama	Algorytm Jarvisa	Algorytm Andrewsa	Algorytm przyrostowy	Algorytm dziel i rządź	Algorytm QuickHull	Algorytm Chana
Zbiór 1	10	10	10	10	10	10	10
Zbiór 2	100	100	100	100	100	100	100
Zbiór 3	8	8	8	8	8	8	8
Zbiór 4	4	4	4	4	4	4	4

Tabela 2: Liczba punktów należących do otoczki zwróconych przez dane algorytmy

Z wyników uzyskanych z Tabela 2 wynika, że liczba punktów otoczki dla każdego zbioru danych oraz dla różnych algorytmów jest taka sama, co jest zgodne z oczekiwaniami. Dodatkowo, porównanie wszystkich wyników w Jupyter Notebook wykazuje, że otoczki składają się z identycznych punktów. Wyniki te są zgodne z oczekiwaniami teoretycznymi. Zbiór 2 składa się ze 100 punktów rozmieszczonych na okręgu, co oznacza, że wszystkie te punkty muszą należeć do obliczonej otoczki wypukłej. Zbiór 3 przedstawia przypadek, w którym punkty znajdują się na bokach prostokąta. Jeśli nie ma punktów w wierzchołkach prostokąta, to na każdym boku powinny znajdować się dokładnie dwa punkty. Zbiór 4 przedstawia sytuację, w której punkty znajdują się na bokach oraz przekątnych

kwadratu, a także w wierzchołkach kwadratu. Oczekiwany wynik jest, że otoczka będzie składać się z 4 punktów leżących na wierzchołkach kwadratu.

8. Wizualizacja

Wszystkie algorytmy znajdujące otoczki wypukłe zostały wyposażone w funkcje wizualizujące ich przebieg i wizualizacje w formie pliku gif. Wizualizacje te zostały przeprowadzone na powyższych zbiorach danych oznaczonych jako 1, 2, 3 i 4, które reprezentowały różnorodne przypadki testowe. Cały proces został zrealizowany w środowisku Jupyter Notebook.

9. Analiza efektywności algorytmów

Na potrzeby porównania złożoności czasowej powyższych algorytmów oraz pomiaru ich czasów wykonywania, wykorzystano zbiory należące do zbiorów z Sekcji 4. Dla ustalonych wartości n :

- Zbiory A, C, D są zbiorem n -punktowymi, gdzie

$$n \in \{1000, 5000, 10000, 50000, 100000, 500000, 1000000\} \quad (9)$$

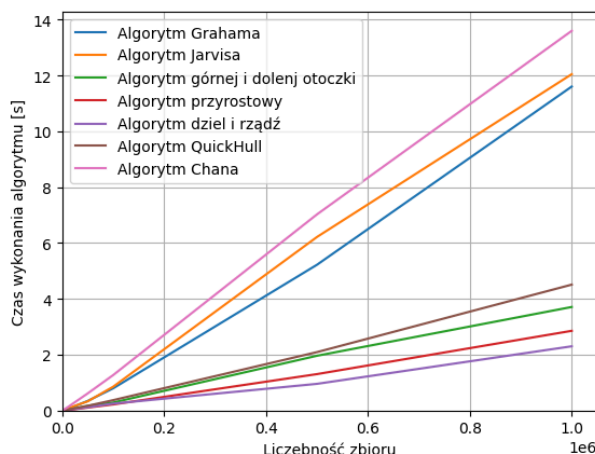
- Zbiory C są zbiorami n -punktowymi, gdzie

$$n \in \{1000, 2000, 3000, 4000, 5000, 6000, 7000, 8000, 9000, 10000\} \quad (10)$$

9.1. Zbiory A

Liczebność zbioru	Czas wykonania [s]						
	Algorytm Grahama	Algorytm Jarvisa	Algorytm górnej i dolnej otoczki	Algorytm przyrostowy	Algorytm dziel i rządź	Algorytm QuickHull	Algorytm Chana
1000	0.0056	0.0082	0.0017	0.0013	0.0019	0.0029	0.0113
5000	0.0236	0.0293	0.0074	0.0067	0.0107	0.0136	0.0564
10000	0.0482	0.0726	0.0178	0.0149	0.0214	0.0292	0.114
50000	0.3455	0.3309	0.1694	0.1048	0.1186	0.1705	0.6159
100000	0.7984	0.8562	0.2873	0.2184	0.2482	0.3728	1.2731
500000	5.2237	6.2191	1.9622	1.3053	0.9559	2.0934	7.0285
1000000	11.607	12.0497	3.7096	2.8555	2.3013	4.5103	13.6028

Tabela 3: Czasy wykonania algorytmów dla zbiorów A n -punktowych



Rysunek 10: Wykresy czasów wykonania algorytmów w zależności od liczebności zbiorów

Z analizy Tabeli 3 oraz Wykresu 10 wynika, że najefektywniejszymi algorytmami w przeprowadzonych badaniach okazały się: algorytm dziel i rządź, algorytm przyrostowy, algorytm QuickHull oraz algorytm górnej i dolnej otoczki. Algorytmy te przewyższyły wydajnością algorytm Grahama, pomimo identycznej złożoności obliczeniowej $O(n \log n)$. Prawdopodobną przyczyną tej różnicy jest mniejsza stała ukryta w notacji O , która odgrywa istotną rolę w praktycznych zastosowaniach dla rozważanych rozmiarów zbiorów danych.

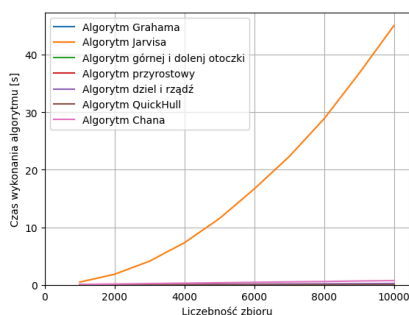
Ponadto, zauważono różnice w wydajności między algorytmem Chana a algorytmem Jarvisa. Oba algorytmy charakteryzują się zależnością czasową od liczby punktów należących do otoczki wypukłej. W przypadku losowego rozkładu punktów, oba algorytmy wykazują najwolniejsze działanie w porównaniu do innych metod. Co jednak zaskakujące, algorytm Chana w zadziałał wolniej niż algorytm Jarvisa, pomimo teoretycznie lepszej asymptotycznej złożoności obliczeniowej. Może to wynikać z większego narzutu obliczeniowego w implementacji algorytmu Chana, który jest bardziej złożony w praktyce.

Warto zauważyć, że funkcja będąca iloczynem funkcji liniowej i logarytmicznej charakteryzuje się w praktyce podobnym przebiegiem do funkcji liniowej.

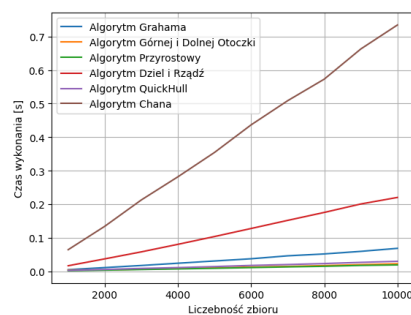
9.2. Zbiory B

Liczebność zbioru	Czas wykonania [s]						
	Algorytm Grahama	Algorytm Jarvisa	Algorytm górnej i dolnej otoczki	Algorytm przyrostowy	Algorytm dziel i rządź	Algorytm QuickHull	Algorytm Chana
1000	0.0049	0.4691	0.004	0.0017	0.0166	0.0027	0.0645
2000	0.0111	1.8327	0.004	0.0035	0.0371	0.0055	0.1343
3000	0.0174	4.109	0.0063	0.0053	0.058	0.0086	0.2129
4000	0.0241	7.3129	0.0087	0.0072	0.0805	0.0111	0.2819
5000	0.0308	11.5007	0.0105	0.009	0.1037	0.0141	0.3545
6000	0.0374	16.6623	0.0128	0.0111	0.1277	0.0176	0.4367
7000	0.0462	22.2859	0.0146	0.0132	0.1519	0.0202	0.5088
8000	0.0518	28.8509	0.0171	0.015	0.1756	0.0231	0.5731
9000	0.0595	36.7158	0.0199	0.0176	0.2008	0.0267	0.6631
10000	0.0685	45.0405	0.0226	0.0191	0.2203	0.0298	0.7346

Tabela 4: Czasy wykonania algorytmów dla zbiorów B n -punktowych



Rysunek 11: Wykresy czasów wykonania algorytmów w zależności od liczebności zbiorów



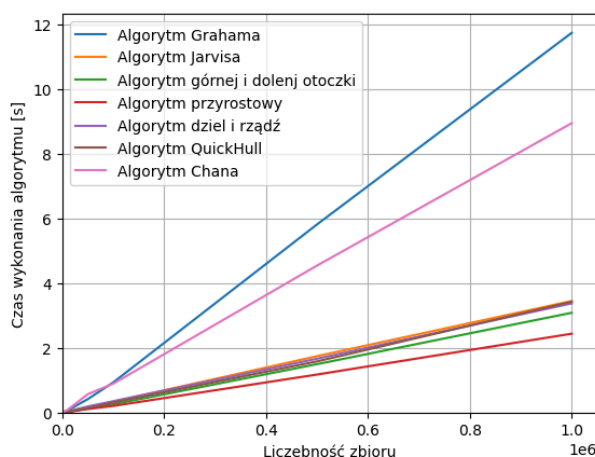
Rysunek 12: Wykresy czasów wykonania algorytmów w zależności od liczebności zbiorów

Z Tabeli 4, Rynku 11 oraz 12, wynika, że jest to przypadek pesymistyczny dla algorytmów, których złożoność obliczeniowa zależy od liczby punktów należących do otoczki wypukłej, oznaczonej jako h . W przypadku punktów rozmieszczonych na okręgu $h = n$. Algorytm Jarvisa osiąga złożoność $O(n^2)$, podczas gdy algorytm Chana ma złożoność $O(n \log n)$, co sprawia, że radzi sobie lepiej w sytuacji pesymistycznej (widać to na Rysunku 11). Najszybciej zadziałały algorytmy: algorytm przyrostowy, algorytm górnej i dolnej otoczki oraz algorytm QuickHull. Jednak z tą różnicą, że algorytm Grahama okazał się szybszy od algorytmu Dziel i rządź.

9.3. Zbiory C

Liczebność zbioru	Czas wykonania [s]						
	Algorytm Grahama	Algorytm Jarvisa	Algorytm górnej i dolnej otoczki	Algorytm przyrostowy	Algorytm dziel i rządź	Algorytm QuickHull	Algorytm Chana
1000	0.0038	0.0039	0.002	0.0018	0.0035	0.0026	0.0076
5000	0.0293	0.0168	0.009	0.0074	0.0162	0.0107	0.0397
10000	0.0604	0.0311	0.0163	0.0144	0.0316	0.0222	0.0797
50000	0.4147	0.1673	0.1298	0.1084	0.1871	0.1361	0.5671
100000	0.933	0.3373	0.2536	0.2037	0.351	0.309	0.8931
500000	5.8178	1.7432	1.4959	1.1748	1.6647	1.5813	4.5444
1000000	11.7422	3.4494	3.0847	2.4376	3.3749	3.4272	8.9493

Tabela 5: Czasy wykonania algorytmów dla zbiorów C n -punktowych



Rysunek 13: Wykresy czasów wykonania algorytmów w zależności od liczebności zbiorów

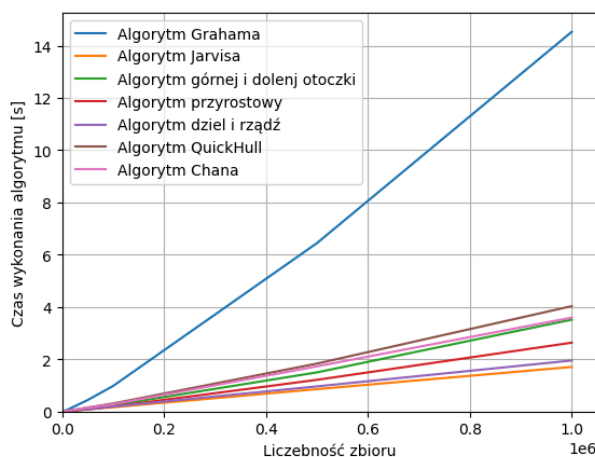
Analizując wyniki przedstawione w Tabeli 5 oraz Wykresie 13, można stwierdzić, że najszybciej zadziałały następujące algorytmy: algorytm przyrostowy, algorytm dolnej i górnej otoczki, algorytm dziel i rządź oraz algorytm QuickHull.

Kolejnym najszybszym algorytmem był algorytm Jarvisa. W tym przypadku liczba punktów należących do otoczki jest ograniczona z góry przez $h = 8$ (na każdym boku prostokąta znajdują się dwa punkty otoczki), co sprawia, że algorytmy Jarvisa i Chana przyjmują optymistyczną złożoność $O(n)$, ponieważ ich czas działania zależy od parametru h . Pomimo tego, algorytm przyrostowy, mimo gorszej złożoności $O(n \log n)$, okazał się szybszy w praktyce. Algorytm Chana okazał się jednym z najwolniejszych algorytmów, pomimo swojej liniowej złożoności. Ponadto zadziałał wolniej niż algorytm Jarvisa, który ma złożoność $O(nh)$.

Algorytm Grahama okazał się najwolniejszy w przypadku, gdy liczba punktów należących do otoczki była ograniczona.

Liczebność zbioru	Czas wykonania [s]						
	Algorytm Grahama	Algorytm Jarvisa	Algorytm górnej i dolnej otoczki	Algorytm przyrostowy	Algorytm dziel i rządź	Algorytm QuickHull	Algorytm Chana
1000	0.0058	0.0015	0.0016	0.0013	0.0015	0.0019	0.0032
5000	0.0333	0.0082	0.0082	0.0091	0.0106	0.0106	0.0137
10000	0.0678	0.0163	0.0151	0.0145	0.019	0.0247	0.0278
50000	0.442	0.0851	0.1103	0.083	0.0916	0.1514	0.149
100000	0.983	0.1646	0.2406	0.1918	0.1918	0.3081	0.2808
500000	6.4466	0.8575	1.4984	1.2135	0.9623	1.8349	1.7299
1000000	14.5287	1.7052	3.516	2.6344	1.9526	4.0328	3.5937

Tabela 6: Czasy wykonania algorytmów dla zbiorów D n -punktowych



Rysunek 14: Wykresy czasów wykonania algorytmów w zależności od liczebności zbiorów

Analizując wyniki przedstawione w Tabeli 6 oraz Wykresie 14, można zauważyć, że w przypadku zbiorów typu C, gdzie liczba punktów należących do obliczanej otoczki wynosi $h = 4$, mamy do czynienia z optymistycznym przypadkiem dla algorytmów Jarvisa i Chana. W takich warunkach obydwa algorytmy osiągają złożoność $O(n)$, co teoretycznie powinno zapewnić im przewagę wydajnościową. Rzeczywiście, algorytm Jarvisa okazał się najszybszy w tym przypadku. Kolejne miejsca pod względem szybkości zajęły algorytm dziel i rządź, algorytm górnej i dolnej otoczki oraz algorytm Chana.

Zaskakujące jest jednak to, że algorytm Chana, mimo swojej liniowej złożoności w tych warunkach, zadziałał wolniej od algorytmów o złożoności $O(n \log n)$, takich jak algorytm dziel i rządź czy algorytm górnej i dolnej otoczki. Dodatkowo, algorytm Jarvisa również zadziałał szybciej niż algorytm Chana, co sugeruje, że algorytm Chana charakteryzuje się większą stałą w złożoności czasowej, która w praktyce wpływa na jego wydajność przy ograniczonej liczbie punktów na otoczce.

10. Wnioski

- Algorytm Grahama okazał się być jednym z najwolniejszych algorytmów w analizowanych zbiorach. Szczególnie niekorzystnie wypadł, gdy liczba punktów w otoczce była ograniczona z góry przez stałą.

- Algorytm przyrostowy okazał się być jednym z najwydajniejszych algorytmów niezależnie od rozważanego zbioru.
- Algorytm dziel i rządź sprawdził się najlepiej spośród wszystkich algorytmów na zbiorach o całkowicie losowym rozkładzie punktów.
- Algorytm Chana, mimo teoretycznej $O(n \log h)$ złożoności okazał się wolniejszy od algorytmu Jarvisa dla zbiorów A, C oraz D, pomimo że Jarvis charakteryzuje się gorszą złożonością $O(nh)$. Wyjątkiem był przypadek pesymistyczny, dla algorytmu Chana i Jarvisa, który występuje w Zbiorach B, gdy $h = n$. W takim przypadku oba algorytmy działają najwolniej i ich złożoności obliczeniowe wynoszą odpowiednio $O(n \log n)$ dla Algorytmu Chana i $O(n^2)$ dla algorytmu Jarvisa.
- W zbiorze C pomimo faktu, że liczba punktów należących do otoczki wypukłej jest ograniczona przez stałą, co pozwala algorytmom Chana oraz Jarvisa osiągać złożoność $O(n)$, w praktyce działały wolniej niż niektóre algorytmy o złożoności $O(n \log n)$.